
AI-Enhanced F1 Simulator

Individual Report

Author:

SONGYUN HAN

Supervisor:

RUZANNA CHITCHYAN



Department of Computer Science

UNIVERSITY OF BRISTOL

A report submitted to the University of Bristol in accordance with the requirements of the degree of MSc Computer Science in the Faculty of Engineering.

SEPTEMBER 2026

Word count: 4289

Table of Contents

	Page
1 Introduction and Context	1
2 Innovation and Creativity	1
2.1 Design Challenges and Initial Decisions	1
2.2 Initial Modular Architecture and Telemetry Interface	1
2.3 Architecture Evolution and Shared Services	2
3 Technical Development	3
3.1 Technical Challenge and Design Rationale	3
3.2 Stateful Event Interpretation and Behavioural Rules	4
3.3 Event Selection and the Granite Boundary	5
3.4 System Integration and Technical Outcome	5
4 Evaluation Skills	6
4.1 Evaluation of the Telemetry Interface	6
4.2 Evaluation Methodology and Collaboration	6
4.3 Test Case Design and Integration Testing	6
4.4 Evaluation Results and Interpretation	7
5 Project Management Skills	8
6 Critical Self-Evaluation and Future Work	9
6.1 Critical Self-Evaluation	9
6.2 Future Work	10
A Additional Architecture Diagram	11

1 Introduction and Context

This project was conducted in collaboration with IBM to investigate the application of AI in race simulators. It aimed to create an immersive and intelligent experience through generative AI. The target users were individuals without professional racing training, who may find it difficult to interpret complex telemetry data while controlling a vehicle. Additional visual and cognitive demands may further increase their cognitive workload [1]. In accordance with the project brief, our team developed four features using the open-source simulator TORCS [2] and IBM Granite [3]: an AI Race Engineer, a Telemetry Coach, Live Commentary, and an AI Racing Bot.

My main responsibility was Live Commentary. I designed and implemented the initial `CommentaryEngine`. It maintained recent telemetry, compressed sensor data, detected and selected race events, controlled repetitive output, and created the initial structured event payload for Granite. It solved the mismatch between high-frequency telemetry and slower model inference by selecting meaningful events before calling the language model. I also implemented the player-vehicle telemetry interface and created the core shared `TelemetryService` and `ModelBroker`. This work shows how deterministic processing can connect high-frequency system state with generative models while improving traceability and responsiveness.

2 Innovation and Creativity

2.1 Design Challenges and Initial Decisions

At the beginning of the project, our team voted to implement all four features. Although they shared the same TORCS environment, they required different inputs, processing methods and outputs. If the feature pipelines depended directly on one another, a change in one pipeline could disrupt the others. The architecture therefore needed to limit these dependencies so that the four features could be developed in parallel.

To meet this need, I designed and presented an initial high-level architecture. At that stage, many implementation details were still unknown. Defining every service and interaction in advance could therefore have delayed development and required frequent changes as the features evolved. I instead defined a small set of boundaries that separated the main responsibilities while allowing each feature owner to begin work.

No teammate proposed an alternative architecture or requested changes to the diagram, so our team used it as the initial structure for development. However, the absence of objections did not constitute formal validation. It provided a practical start, but did not fully define shared services or integration contracts.

2.2 Initial Modular Architecture and Telemetry Interface

Figure 2.1 shows the initial architecture. The four layers had the following responsibilities:

1. The first layer acquired telemetry from TORCS.
2. The second layer processed the telemetry in separate feature pipelines.
3. The third layer provided Granite inference where required, while deterministic and time-critical processing remained local.
4. The final layer presented information to the user or sent control commands back to TORCS.

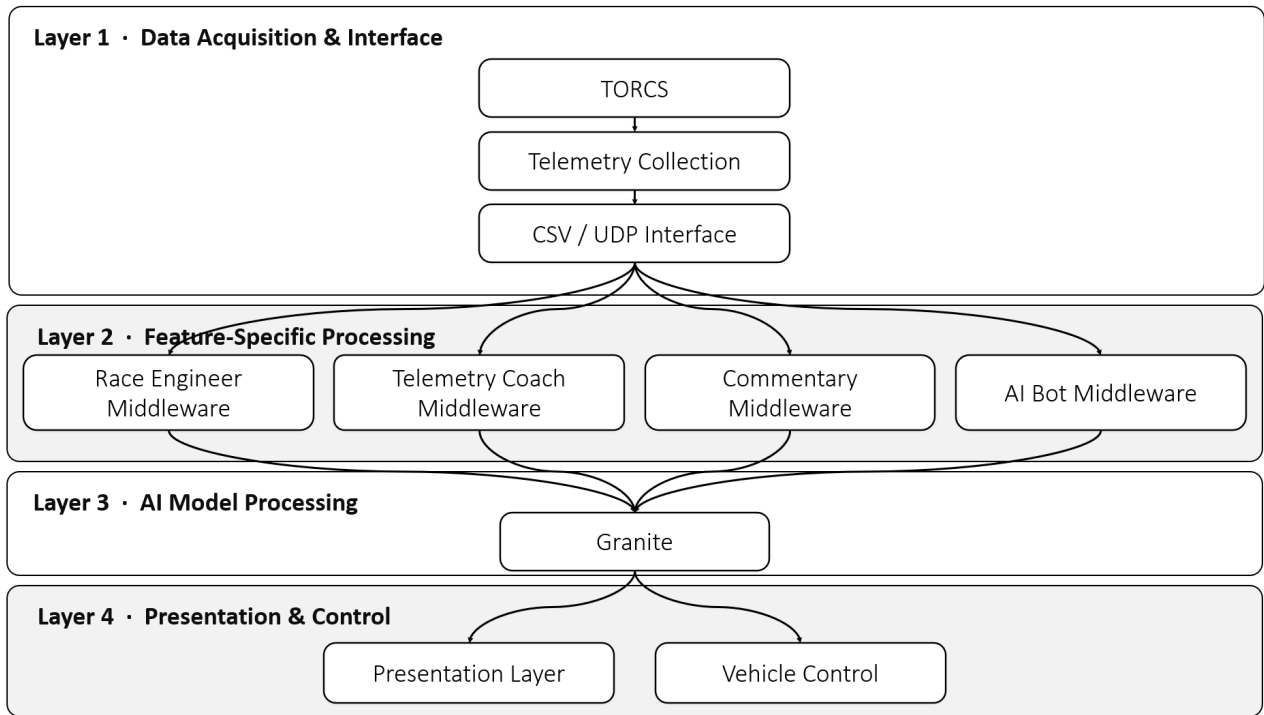


Figure 2.1: Initial four-layer architecture separating telemetry acquisition, feature-specific processing, AI model processing, and presentation or vehicle control.

The architecture separated telemetry acquisition, feature processing, model inference, and presentation or control into different layers. It was intended to reduce dependencies among the four feature pipelines and prevent slower model inference from blocking telemetry collection or time-critical vehicle control.

For the first layer, I created a read-only telemetry interface for player-controlled vehicle. The project simulator was based on a patched TORCS 1.3.7 that included an SCR server [6]. The provided SCR interface required clients to receive sensor data and return control commands synchronously [2]. This was unsuitable for observation-only features, which only needed telemetry. I modified the human driver to emit telemetry without changing the keyboard controls. UDP provided low-overhead live communication, while CSV supported persistent inspection during early development.

I later added race-order packets for Commentary. Commentary, Race Engineer and Telemetry Coach then used this interface, while AI Racing Bot retained its separate SCR control loop. This allowed the three observation-based features to begin development without depending on one another’s internal implementation. However, the initial architecture did not define shared telemetry storage or model-access services. The resulting duplication was discovered during integration and motivated the architecture changes described in the next subsection.

2.3 Architecture Evolution and Shared Services

Integrating the feature pipelines exposed duplicated telemetry and model-access infrastructure, including UDP parsing, race-state storage, model configuration, timeout handling, and failure handling. To remove this duplication, I implemented shared telemetry and model-access services.

The first shared component was `TelemetryService`, which owned the player-telemetry UDP receiver and updated a common `TelemetryStore` containing the latest state and recent history. I first implemented

TelemetryStore for Commentary. During integration, it became the shared telemetry state layer for Commentary, the Race Engineer, and the Telemetry Coach. These features then used the same representation without maintaining separate receivers and state histories.

The second component was ModelBroker, which provided priority scheduling, request timeouts, stale-request invalidation, and queue eviction. This allowed all four features to share Granite access while telemetry acquisition, local feature processing, and vehicle control continued independently of model latency.

The resulting architecture balanced feature autonomy with shared infrastructure. Separate pipelines preserved autonomy but duplicated common services. Full centralisation increased the risk of coupling between feature owners. It shared telemetry acquisition, state storage, model access and runtime coordination while keeping feature-specific logic separate.

I implemented the core TelemetryService, ModelBroker, and initial unified runtime in refactoring. My teammates and I then integrated features with these shared interfaces and extended the corresponding logic on this foundation. My contribution therefore established the core shared infrastructure, while its adoption across features and the resulting final architecture were collaborative outcomes, as shown in Appendix Figure A.1.

The presentation layer also changed during development. Junjian Cao first developed an HTML subtitle interface, after which I experimented with an Electron overlay for Commentary. The Electron prototype was abandoned before final system integration because its high maintenance cost. During later integration, Shangqing Li created a unified HTML dashboard for all four features. I adapted the shared runtime and Commentary components to this interface, after which our team integrated and extended the dashboard.

3 Technical Development

3.1 Technical Challenge and Design Rationale

My biggest contribution to Live Commentary was the initial pipeline. It transformed continuous telemetry into selected race events for natural-language generation. The main challenge was the difference in speed between the simulator and the language model. The player-vehicle interface produced telemetry at approximately 20 Hz, while Granite took much longer to produce a response. Sending every frame risked creating redundant requests, increasing competition for the shared model and generating commentary about outdated race states.

A single telemetry frame was also insufficient for identifying race events. For example, a frame showing race position three did not indicate whether the driver remained third or moved from fourth to third. I therefore treated Commentary as a stateful interpretation problem. The system compared observations over time, recognised a relevant transition, selected one event and then requested natural-language generation.

I compared two approaches. The first sent raw telemetry to Granite and asked the model to identify and describe events. This reduced local processing but made event decisions harder to reproduce, test and trace. It also risked larger and more frequent model inputs. The second used deterministic local rules to detect events and used Granite only to express them in natural language. I selected the second approach because it kept timing-sensitive decisions in inspectable code and gave Granite a smaller, clearer task [7].

I implemented this approach in the initial stateful CommentaryEngine. My work included a bounded telemetry window, temporal comparison, semantic sensor compression, rule-based event detection, priority arbitration, cooldown and repetition control, and the first structured event payload. During later collaboration, other members refined event-specific payload fields, prompt construction, generation and playback behaviour,

while my initial engine remained responsible for converting telemetry into an eligible event.

3.2 Stateful Event Interpretation and Behavioural Rules

The Commentary engine selected telemetry frames from the preceding six seconds using their simulation timestamps. From each window, it derived changes in speed and damage, mean speed, and nearest-opponent distance. It also retained the previous lap count, race position, damage value, and `was_off_track` flag across updates. This context allowed the engine to distinguish a continuing condition from the onset of a new event.

For example, a move from fourth to third place produced `position_change`, whereas an increased lap count produced `lap_complete`. An `off_track` event was emitted when the vehicle moved from $|\text{trackPos}| \leq 1.0$ to $|\text{trackPos}| > 1.0$. The stored flag then prevented subsequent off-track frames from being treated as new events. The rules therefore represented transitions between earlier and current observations rather than classifications of isolated frames.

The thresholds, priorities and cooldowns were chosen pragmatically during prototype development rather than calibrated using an independently labelled dataset. Their purpose was to establish an operational and traceable telemetry-to-commentary pipeline: given the same current and retained observations, the detector produced the same event candidate. Individual parameters could also be revised without redesigning the interpretation process. Table 3.1 therefore specifies my initial implementation rather than claiming that its values were optimal measures of racing significance.

Table 3.1: Heuristic event rules in the initial Commentary implementation.

Event	Trigger or state transition	Priority	Cooldown
<code>contact</code>	Damage increase ≥ 5.0 telemetry units	5	1.0 s
<code>position_change</code>	Current race position differs from the stored position	5	1.0 s
<code>off_track</code>	$ \text{trackPos} > 1.0$ after the vehicle was previously on track	5	1.2 s
<code>lap_complete</code>	Current lap count exceeds the stored lap count	4	1.0 s
<code>battle</code>	Front gap < 10.0 m while speed > 60.0 km/h	4	2.5 s
<code>pace_surge</code>	Consecutive speed increase > 22.0 km/h with throttle > 0.8	3	2.5 s
<code>pace_update</code>	The configured baseline interval has elapsed	1	6.0 s

Later collaborative development refined some detection behaviour, but the table records the rules and parameters in my original engine.

The raw telemetry contained 19 track-edge and 36 opponent sensor readings. Passing both arrays directly to Granite would have enlarged the prompt without clearly expressing their racing significance. I compressed the track-edge readings into left, centre, right and tightest openings, and the opponent readings into front, left, right, rear and nearest gaps. This preserved the coarse spatial relationships needed for brief commentary while omitting unused fine-grained angular detail.

The implementation was stateful, but I did not define or implement a formal finite-state machine. Each detector compared incoming telemetry with stored observations and applied a guarded transition rule. This allowed several candidate event types to be recognised in one update, but states and transitions were not represented centrally. Retrospectively, small event-specific state machines or transition tables could have made guards and expected behaviour more explicit and easier to test. A global machine would require composite states for overlapping events, while parallel machines would require coordination. Priority arbitration would still be necessary when several events became eligible together.

3.3 Event Selection and the Granite Boundary

Several rules could fire within the same telemetry window. Sending every result to Granite would create overlapping commentary and extra model calls; a collision might also involve a lost position and a nearby car. I handled this by treating detections as candidates and passing only the highest-priority eligible one to Granite, using the ordering in Table 3.1. Urgent incidents and major state changes could therefore displace routine updates, while the model received one event at a time.

Priority did not solve repetition on its own. I added a global wall-clock cooldown, event-specific simulation-time cooldowns and a recent-text check after normalisation. Each control addressed a different problem: overall frequency, repeated event signatures or similar generated wording. The two clocks remained separate because TORCS may be paused or run at a different simulation rate. Ten seconds for the user is not always ten seconds of race progress.

The engine could also run in event, interval or hybrid mode. Event mode admitted only detected events. Interval mode provided periodic race context, while hybrid mode combined both behaviours but treated periodic updates as lower priority. Changing mode altered the selection policy rather than the detector implementations, allowing the same event rules to support different commentary patterns.

Once an event passed the priority and repetition checks, I converted it into a bounded structured payload. The initial version recorded the event type, detection reason and simulation time, then added compact evidence about the current state, recent changes and nearby cars. It also specified the language, tone and maximum output length. Later collaborative work replaced parts of this general context with fields tailored to individual event types.

Take position change as an example. Suppose the stored state contains race position four and the latest frame contains position three. The local engine generates a `position_change` candidate with priority 5 and records the move to P3. If no higher-priority event replaces it, the engine creates a structured payload with the event and relevant race context. Granite then converts this evidence into a concise commentary sentence. Granite does not decide whether an overtake occurred. Deterministic code makes that decision before language generation [8].

This boundary improved traceability. For an incorrect position-change event, developers can inspect the race-position transition and stored state. For an omitted event, they can examine its threshold, cooldown, signature or competing priority. If the event was correct but the wording was poor, the problem lay after the payload boundary in prompt construction or language generation. Separating these responsibilities therefore made failures easier to locate than an end-to-end prompt containing raw telemetry.

3.4 System Integration and Technical Outcome

During integration, the independently developed pipelines were found to duplicate UDP reception, telemetry storage and model-access logic. I implemented the core shared `TelemetryService`, `TelemetryStore`, `ModelBroker` and initial runtime, then migrated `Commentary` from its independent entry point to these interfaces. The `CommentaryEngine` retained temporal interpretation, event detection, arbitration and repetition control, while shared services handled telemetry reception and asynchronous Granite access. Local detection could therefore continue while an earlier model request was processed.

My contribution established the shared foundation and initial `Commentary` integration. Other members subsequently connected their feature logic and completed the dashboard, event-specific payloads, and speech and playback behaviour. The late refactor also showed that feature independence had continued too long without

stable shared contracts. In future, I would review telemetry, model-access and presentation interfaces earlier and test one cross-feature path before prolonged parallel development.

4 Evaluation Skills

4.1 Evaluation of the Telemetry Interface

I evaluated the player-vehicle telemetry interface as I developed it. In TORCS, I applied controlled steering, throttle and braking inputs while logging speed, control values, lap progress, race position and other observable fields. Each value was expected to respond consistently to the corresponding action without interfering with the existing keyboard controls. The CSV records showed the expected changes, while manual control remained operational throughout.

4.2 Evaluation Methodology and Collaboration

My main evaluation contribution was the methodology and implementation specification for Live Commentary. I based it on Junjie Peng’s project-wide testing framework, which distinguished dependency-free tests, integration tests, external-service checks and end-to-end acceptance. It also made a limitation clear: a successful demonstration, or even a large number of passing tests, says little unless the environment, expected outcome and acceptance boundary are defined.

I organised the Commentary evaluation into four work packages, each addressing a different pipeline risk. Functional correctness examined deterministic event rules and repetition controls. Detection accuracy considered missed and misclassified incidents; latency measured whether commentary arrived soon enough to remain useful. Stability and recovery covered continuous telemetry operation and failures involving Granite, WebSocket clients and TTS.

For each package, I specified the test cases, data formats, matching rules, acceptance thresholds and reporting requirements. Junjian Cao implemented and ran most automated tests and analysis scripts, although some materials I authored were committed through his account. Junjie Peng added the supporting t_0 – t_5 latency instrumentation. My specification set the expected behaviour for event detection, priority, cooldown and deduplication; their implementation work tested those expectations and uncovered assumptions and defects. Evaluation was therefore collaborative, but our responsibilities remained identifiable.

4.3 Test Case Design and Integration Testing

I divided Commentary into functional units covering input processing, event detection, operating modes, priority selection, cooldown, deduplication, runtime integration and fault handling. For input processing, the equivalence partitions covered complete valid telemetry, empty input, missing optional fields, malformed values and repeated frames. Each mapped to a distinct expected response: normal processing, safe defaulting, controlled rejection or duplicate-event suppression.

For threshold-based rules, I used three-point boundary-value analysis, selecting one value below, one at and one above each implemented boundary. The `contact` rule, for example, triggered when the TORCS `damage` field increased by at least 5.0. Increases of 4.99, 5.00 and 5.01 were therefore expected to produce no event, `contact` and `contact`; all three tests behaved as specified. The `off_track` rule instead required $|\text{track_pos}| > 1.0$.

Values of 0.999 and 1.000 produced no event, while 1.001 produced `off_track`. These results also matched the expected boundary behaviour.

I also specified temporal sequences rather than testing only isolated frames. These included an unchanged versus incremented lap counter, entering versus remaining outside the track boundary, repeated event signatures within and after cooldown, and simultaneous candidates with different priorities. These were state-transition-style tests over the implemented stateful rules, although the detector itself was not a formal finite-state machine.

Integration tests evaluated behaviour at user-visible boundaries. One requirement was to suppress repeated model text before it reached the dashboard. Testing at the WebSocket consumer boundary exposed a broadcast-before-deduplication defect: users could see the repeated text before the backend rejected it. Junjian Cao corrected the ordering, and the regression test then passed. Testing only the internal cache was insufficient to verify the required user-visible behaviour.

4.4 Evaluation Results and Interpretation

Table 4.1 summarises the resulting evidence and the limits of the conclusions that can be drawn.

Table 4.1: Summary of Live Commentary evaluation evidence, results and limitations.

Evaluation area	Evidence and result	Limitation
Automated testing	All 74 functional and integration tests passed, including 29 event-boundary and nine cooldown/deduplication tests.	The suite demonstrated specified behaviour under predefined conditions but did not provide code-coverage evidence.
Event detection	Six sessions on two tracks produced 138 reference events, of which 132 were matched: precision 1.000, recall 0.957 and F1 score 0.978.	The reference labels for several categories were partly derived by reviewing detector output and were therefore not fully independent.
Off-track detection	8 of 14 reference events were detected, giving a recall of 0.571.	Some valid events were not reported when simultaneous candidates competed for one output.
Latency	Across 48 triggers, median detection, first-token, complete-response and caption-display times were 0.479, 0.746, 1.403 and 1.919 seconds.	Detection P95 was 2.635 seconds and two triggers lacked a captured caption-display result, showing a substantial tail beyond the median.
Endurance testing	Two 30-minute runs recorded 565 events and 402 successful outputs, with no model failures, duplicate displays or unhandled exceptions.	The 71.15% output rate was affected by request pre-emption, and two runs do not establish long-term reliability.
Fault recovery	Twelve conditions were repeated five times; nine passed every trial and three produced partial results. No middleware crashes occurred.	Granite unavailability, WebSocket disconnection and telemetry interruption were not independently shown to recover in their fault-injection steps.

For detection accuracy, I designed a protocol using a one-second matching tolerance across six driving sessions on two tracks, while other team members executed the sessions. The aggregate figures indicate that most configured events were detected, but they should not be treated as an unbiased estimate of real-world accuracy because the reference annotations were partly informed by system output. The low `off_track` recall was particularly important: it showed that passing boundary tests did not guarantee that an event would be reported when the priority policy allowed only one simultaneous candidate to proceed.

For the latency study, I defined six timestamps: telemetry arrival (t_0), event detection (t_1), first model token (t_2), completed model response (t_3), caption display (t_4) and TTS playback (t_5). Junjie Peng implemented the instrumentation. The median results were compatible with live use, yet the detection P95 reached 2.635

seconds. This tail shows how a median can conceal delayed cases. The results therefore provide preliminary operational evidence, not proof that every event arrived promptly.

I contributed the R02 endurance dataset directly. Over 1,802 seconds, the system detected 249 events and produced 188 successful outputs. No model failures, duplicate displays or unhandled exceptions were recorded. The gap between requests and successful outputs was consistent with pre-emption in interrupt mode; however, the run could not establish that pre-emption accounted for every missing output. Junjian Cao and other members of our team contributed the remaining test execution, result collection and fault-recovery evidence.

Taken together, the results support functional conformance for the specified cases, strong exploratory detection performance for most event categories and stable operation in the recorded runs. They do not establish software quality as a whole or fitness for use. Partially self-referential ground truth, weak `off_track` recall, the latency tail, limited endurance duration and incomplete fault-recovery evidence restrict the technical conclusions. Nor did a user study assess the commentary’s relevance, naturalness or timing. A stronger evaluation would define independent reference events in advance, cover more tracks and users, and combine objective measurements with user assessment.

5 Project Management Skills

I managed my workload by ordering tasks according to their functional dependencies and implementing them incrementally. Commentary, the Race Engineer and the Telemetry Coach all relied on the player-vehicle telemetry interface, so I implemented it first and committed it as one of the earliest contributions to the repository. I then began developing Commentary, dividing the work into stages spanning telemetry processing, event detection, model generation and user-facing output. We first established a working end-to-end pipeline and then expanded it incrementally.

After establishing the first end-to-end pipeline, the Commentary team continuously refined the prompts, user interface and voice features through small, iterative steps. Following each round of modifications, we ran the full pipeline to verify its functionality, adjusted the implementation based on the observed issues and proceeded to the next round only after validating the results. With this method, we ensured that a functional version of the system remained available throughout the iteration process, while new capabilities and bug fixes were continuously integrated in small increments. This process embodied Agile development principles centred on working software, feedback and adaptive adjustment.

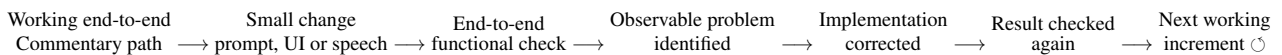


Figure 5.1: Feedback-driven incremental development after establishing the first end-to-end Commentary path.

As Commentary neared completion, I reprioritised the remaining tasks, shifting my focus from further expanding Commentary to supporting the migration to a shared architecture. System integration exposed that telemetry reception, state storage and model access had been redundantly implemented across different features. Junjian Cao first consolidated telemetry parsing and prompt assembly within Commentary, after which Jian Ye defined a migration plan and integration contract for the gradual adoption of a shared runtime without blocking parallel feature development. I then implemented the core components—`TelemetryService`, `TelemetryStore`, `ModelBroker` and an initial unified runtime—and migrated Commentary to these interfaces. Teammates integrated their features with the shared infrastructure. This adjustment allowed me to prioritise resolving integration dependencies that affected multiple features rather than continuing to add capabilities that

served only Commentary.

These redundant implementations were not discovered until late-stage integration. This meant that, although the incremental checks ensured that Commentary remained functional, the feedback was largely confined to a single feature and failed to expose incompatible assumptions across the system in a timely manner. In future projects, I will establish a streamlined cross-feature integration path earlier and review shared interface contracts to identify and resolve incompatibilities while the cost of modification remains low.

I also established and organised a shared knowledge base in Feishu. All members maintained the weekly progress page and consulted the development plans, technical instructions, interface descriptions and architecture records, while Mingyu Yang uploaded meeting recordings. The space therefore provided a shared reference for progress tracking, implementation and integration. However, access and consultation did not guarantee that every technical decision was understood or applied consistently. Feishu was effective for making information available and visible, but it needed more explicit review responsibilities to establish shared technical understanding.

Frequent synchronous pair-programming sessions with Junjian Cao provided a more direct coordination mechanism. We inspected event behaviour, resolved defects and coordinated telemetry interpretation, language generation and testing while retaining identifiable responsibility boundaries. In future projects, I would supplement weekly tracking and shared documentation with named reviewers and short technical walkthroughs. This would make shared understanding observable rather than assuming that publishing information was sufficient.

6 Critical Self-Evaluation and Future Work

6.1 Critical Self-Evaluation

My main contributions were the initial player-telemetry interface, the initial stateful Commentary event-interpretation engine and the core shared-service components, including `TelemetryService`, `ModelBroker` and the initial unified runtime. These components established a traceable path from TORCS telemetry to generated commentary. However, the project showed that modular components support parallel development only when our team also agrees on and verifies its shared interfaces.

The evolution of the presentation layer illustrates this limitation. Junjian Cao developed an early HTML-based subtitle interface, while I implemented an Electron overlay for Commentary. Both required repeated modification as backend connections, speech behaviour, configuration and feature outputs evolved between the initial Commentary pipeline and final system integration. The Electron overlay was ultimately abandoned because of its high maintenance cost. During integration, Shangqing Li created the shared HTML dashboard, and our team subsequently integrated and extended it as the final presentation layer. If I repeated the project, I would first agree on a technology-independent output contract, then validate placeholder outputs from each feature through a minimal shared interface before selecting a presentation technology.

The project also changed how I think about communication within our team. Our team used Feishu to record progress, plans, technical instructions and interface descriptions. This gave us a shared reference, but it did not keep the four features moving at the same pace. We did not hold regular progress meetings or use explicit reviews to confirm that decisions had been understood and applied consistently. Pair programming with Junjian Cao was more effective for resolving specific problems because we could question assumptions and identify defects while working on the relevant components. If I repeated the project, I would retain the shared documentation but add

regular reviews, agreed integration milestones and brief technical walkthroughs. Differences in progress and interface interpretation would then become visible earlier.

I also relied on pragmatic event thresholds. During prototyping, I selected initial values to establish the end-to-end pipeline rather than calibrating them against labelled telemetry. They supported a working deterministic detector, but they did not provide validated definitions of racing events, particularly in ambiguous cases. The evaluation had a related limitation: reference labels for five categories were derived partly from the detector’s own output. The reported precision, recall and F1 score [9] therefore describe exploratory performance, not an unbiased estimate of accuracy. If I repeated this work, I would create independently labelled recordings, calibrate the thresholds against them and reserve separate data for the final evaluation.

6.2 Future Work

In future, our team could add continuous integration before feature development diverges. Each pull request would run automated tests and reject changes that break the shared runtime or any feature combination. This would reveal incompatible assumptions while interfaces are changing, rather than during final integration. TORCS and Granite tests could remain manual because they require the simulator and model services.

Commentary needs a narrower behavioural model. Lap completion has a clear transition, whereas ambiguous incidents unfold across several seconds of telemetry. I would describe each detector with a transition table containing its state, input, guard, action and suppression conditions. One global state machine would be cumbersome because events overlap; parallel event-state models would represent this concurrency more clearly. Independently annotated recordings could then support threshold calibration and comparison of a lightweight time-series classifier with the rule-based detector [10].

Generated text needs separate validation. A post-generation check should verify event and car references against the payload and enforce the word limit. If Granite is unavailable, an event-specific template could provide a short factual message. Evaluation would use independent annotators, unseen race conditions and a user study of relevance, naturalness and timing.

A Additional Architecture Diagram

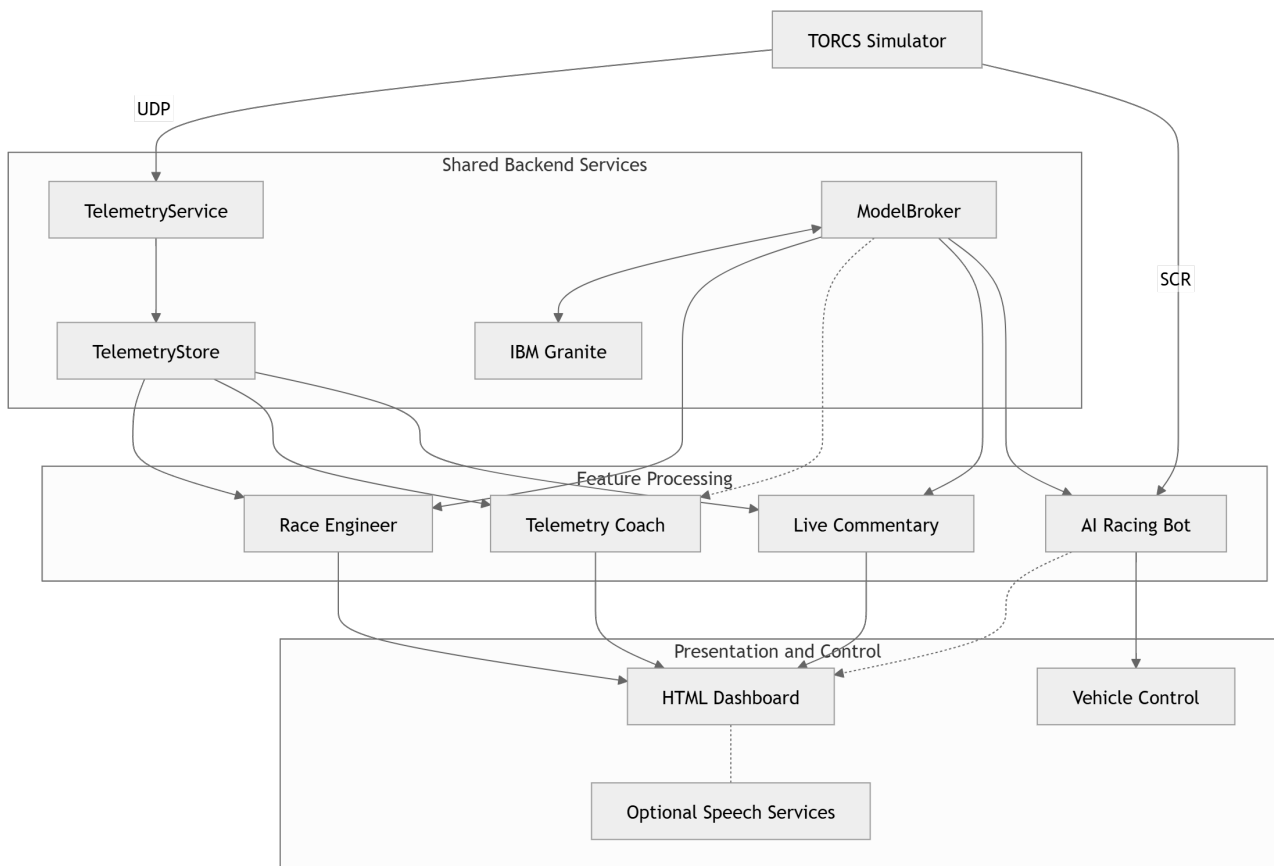


Figure A.1: Final shared-service architecture integrating telemetry distribution, feature-specific processing, model access, speech services, dashboard presentation, and vehicle control.

References

- [1] A. Hamish Jamson and Natasha Merat. Surrogate in-vehicle information systems and driver behaviour: Effects of visual and cognitive load in simulated rural driving. *Transportation Research Part F: Traffic Psychology and Behaviour*, 8(2):79–96, 2005. doi: 10.1016/j.trf.2005.04.002.
- [2] Daniele Loiacono, Pier Luca Lanzi, Julian Togelius, Enrique Onieva, David A. Pelta, Martin V. Butz, Thomas D. Lönneker, Luigi Cardamone, Diego Perez, Yago Sáez, Mike Preuss, and Jan Quadflieg. The 2009 simulated car racing championship. *IEEE Transactions on Computational Intelligence and AI in Games*, 2(2):131–147, 2010. doi: 10.1109/TCIAIG.2010.2050590.
- [3] IBM Granite Team. Granite-4.1-8B model card. Hugging Face, 2026. URL <https://huggingface.co/ibm-granite/granite-4.1-8b>. Accessed: 2026-08-26.
- [4] Tatsuya Ishigaki, Goran Topic, Yumi Hamazono, Hiroshi Noji, Ichiro Kobayashi, Yusuke Miyao, and Hiroya Takamura. Generating racing game commentary from vision, language, and structured data. In *Proceedings of the 14th International Conference on Natural Language Generation*, pages 103–113, Aberdeen, Scotland, UK, 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.inlg-1.11. URL <https://aclanthology.org/2021.inlg-1.11/>.
- [5] Tatsuya Ishigaki, Goran Topić, Yumi Hamazono, Ichiro Kobayashi, Yusuke Miyao, and Hiroya Takamura. Audio commentary system for real-time racing game play. In *Proceedings of the 16th International Natural Language Generation Conference: System Demonstrations*, pages 9–10, Prague, Czechia, 2023. Association for Computational Linguistics. URL <https://aclanthology.org/2023.inlg-demos.4/>.
- [6] fmirus. TORCS 1.3.7 with SCR patch. GitHub repository, n.d. URL <https://github.com/fmirus/torcs-1.3.7>. Accessed: 2026-08-27.
- [7] Xiaoyu Shen, Jun Suzuki, Kentaro Inui, Hui Su, Dietrich Klakow, and Satoshi Sekine. Select and attend: Towards controllable content selection in text generation. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 579–590, Hong Kong, China, 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1054. URL <https://aclanthology.org/D19-1054/>.
- [8] Sam Wiseman, Stuart M. Shieber, and Alexander M. Rush. Challenges in data-to-document generation. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 2253–2263, Copenhagen, Denmark, 2017. Association for Computational Linguistics. doi: 10.18653/v1/D17-1239. URL <https://aclanthology.org/D17-1239/>.
- [9] Marina Sokolova and Guy Lapalme. A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45(4):427–437, 2009. doi: 10.1016/j.ipm.2009.03.002.
- [10] Hassan Ismail Fawaz, Germain Forestier, Jonathan Weber, Lhassane Idoumghar, and Pierre-Alain Muller. Deep learning for time series classification: A review. *Data Mining and Knowledge Discovery*, 33(4): 917–963, 2019. doi: 10.1007/s10618-019-00619-1.